

3. Each attribute community has developed its own vocabulary. The performance community has “events” arriving at a system, the security community has “attacks” arriving at a system, the availability community has “faults” arriving, and the usability community has “user input.” All of these may actually refer to the same occurrence, but they are described using different terms.

A solution to the first two problems (untestable definitions and overlapping issues) is to use *quality attribute scenarios* as a means of characterizing quality attributes (see Section 3.3). A solution to the third problem is to illustrate the concepts that are fundamental to that attribute community in a common form, which we do in Chapters 4–14.

We will focus on two categories of quality attributes. The first category includes those attributes that describe some property of the system at runtime, such as availability, performance, or usability. The second category includes those that describe some property of the development of the system, such as modifiability, testability, or deployability.

Quality attributes can never be achieved in isolation. The achievement of any one will have an effect—sometimes positive and sometimes negative—on the achievement of others. For example, almost every quality attribute negatively affects performance. Take portability: The main technique for achieving portable software is to isolate system dependencies, which introduces overhead into the system’s execution, typically as process or procedure boundaries, which then hurts performance. Determining a design that may satisfy quality attribute requirements is partially a matter of making the appropriate tradeoffs; we discuss design in Chapter 21.

In the next three sections, we focus on how quality attributes can be specified, what architectural decisions will enable the achievement of particular quality attributes, and what questions about quality attributes will enable the architect to make the correct design decisions.

3.3 Specifying Quality Attribute Requirements: Quality Attribute Scenarios

We use a common form to specify all QA requirements as scenarios. This addresses the vocabulary problems we identified previously. The common form is testable and unambiguous; it is not sensitive to whims of categorization. Thus it provides regularity in how we treat all quality attributes.

Quality attribute scenarios have six parts:

- *Stimulus*. We use the term “stimulus” to describe an event arriving at the system or the project. The stimulus can be an *event* to the performance community, a *user operation* to the usability community, or an *attack* to the security community, and so forth. We use the same term to describe a motivating action for developmental qualities. Thus a stimulus for modifiability is a request for a modification; a stimulus for testability is the completion of a unit of development.
- *Stimulus source*. A stimulus must have a source—it must come from somewhere. Some entity (a human, a computer system, or any other actor) must have generated the

stimulus. The source of the stimulus may affect how it is treated by the system. A request from a trusted user will not undergo the same scrutiny as a request by an untrusted user.

- *Response.* The response is the activity that occurs as the result of the arrival of the stimulus. The response is something the architect undertakes to satisfy. It consists of the responsibilities that the system (for runtime qualities) or the developers (for development-time qualities) should perform in response to the stimulus. For example, in a performance scenario, an event arrives (the stimulus) and the system should process that event and generate a response. In a modifiability scenario, a request for a modification arrives (the stimulus) and the developers should implement the modification—without side effects—and then test and deploy the modification.
- *Response measure.* When the response occurs, it should be measurable in some fashion so that the scenario can be tested—that is, so that we can determine if the architect achieved it. For performance, this could be a measure of latency or throughput; for modifiability, it could be the labor or wall clock time required to make, test, and deploy the modification.

These four characteristics of a scenario are the heart of our quality attribute specifications. But two more characteristics are important, yet often overlooked: environment and artifact.

- *Environment.* The environment is the set of circumstances in which the scenario takes place. Often this refers to a runtime state: The system may be in an overload condition or in normal operation, or some other relevant state. For many systems, “normal” operation can refer to one of a number of modes. For these kinds of systems, the environment should specify in which mode the system is executing. But the environment can also refer to states in which the system is not running at all: when it is in development, or testing, or refreshing its data, or recharging its battery between runs. The environment sets the context for the rest of the scenario. For example, a request for a modification that arrives after the code has been frozen for a release may be treated differently than one that arrives before the freeze. The fifth successive failure of a component may be treated differently than the first failure of that component.
- *Artifact.* The stimulus arrives at some target. This is often captured as just the system or project itself, but it’s helpful to be more precise if possible. The artifact may be a collection of systems, the whole system, or one or more pieces of the system. A failure or a change request may affect just a small portion of the system. A failure in a data store may be treated differently than a failure in the metadata store. Modifications to the user interface may have faster response times than modifications to the middleware.

To summarize, we capture quality attribute requirements as six-part scenarios. While it is common to omit one or more of these six parts, particularly in the early stages of thinking about quality attributes, knowing that all of the parts are there forces the architect to consider whether each part is relevant.

We have created a general scenario for each of the quality attributes presented in Chapters 4–13 to facilitate brainstorming and elicitation of concrete scenarios. We distinguish

general quality attribute scenarios—general scenarios—which are system independent and can pertain to any system, from *concrete* quality attribute scenarios—concrete scenarios—which are specific to the particular system under consideration.

To translate these generic attribute characterizations into requirements for a particular system, the general scenarios need to be made system specific. But, as we have found, it is much easier for a stakeholder to tailor a general scenario into one that fits their system than it is for them to generate a scenario from thin air.

Figure 3.1 shows the parts of a quality attribute scenario just discussed. Figure 3.2 shows an example of a general scenario, in this instance for availability.

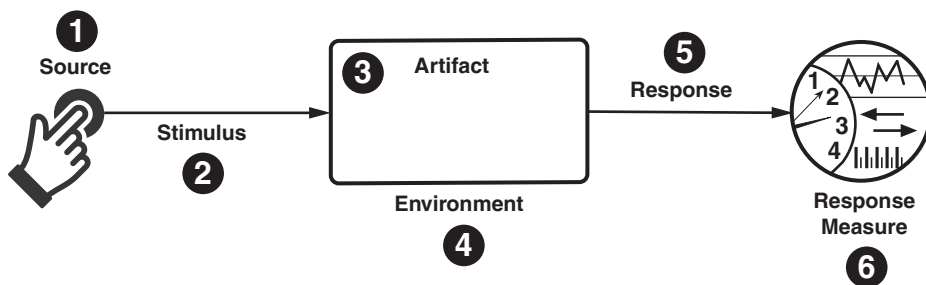


FIGURE 3.1 The parts of a quality attribute scenario

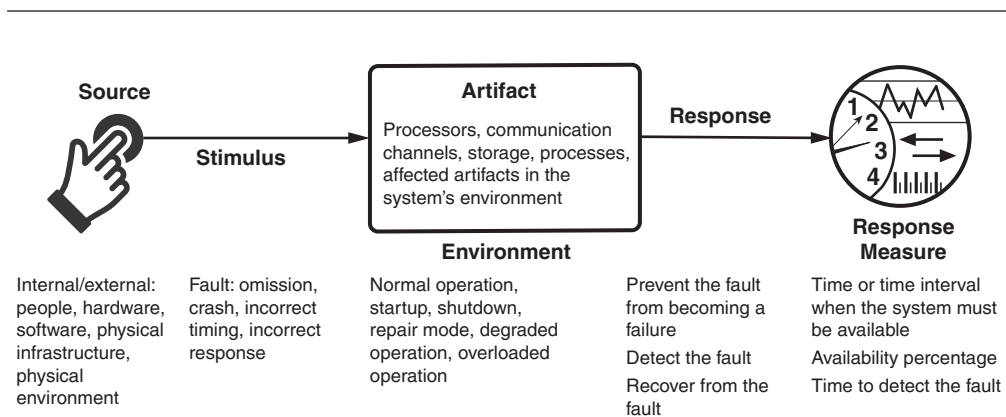


FIGURE 3.2 A general scenario for availability

Not My Problem

Some time ago I was doing an architecture analysis on a complex system created by and for Lawrence Livermore National Laboratory. If you visit this organization's website (llnl.gov) and try to figure out what Livermore Labs does, you will see the word "security" mentioned over and over. The lab focuses on nuclear security, international and domestic security, and environmental and energy security. Serious stuff . . .

Keeping this emphasis in mind, I asked my clients to describe the quality attributes of concern for the system that I was analyzing. I'm sure you can imagine my surprise when security wasn't mentioned once! The system stakeholders mentioned performance, modifiability, evolvability, interoperability, configurability, and portability, and one or two more, but the word "security" never passed their lips.

Being a good analyst, I questioned this seemingly shocking and obvious omission. Their answer was simple and, in retrospect, straightforward: "We don't care about it. Our systems are not connected to any external network, and we have barbed-wire fences and guards with machine guns."

Of course, *someone* at Livermore Labs was very interested in security. But not the software architects. The lesson here is that the software architect may not bear the responsibility for every QA requirement.

—RK

3.4 Achieving Quality Attributes through Architectural Patterns and Tactics

We now turn to the techniques an architect can use to *achieve* the required quality attributes: architectural patterns and tactics.

A tactic is a design decision that influences the achievement of a quality attribute response—it directly affects the system's response to some stimulus. Tactics may impart portability to one design, high performance to another, and integrability to a third.

An architectural pattern describes a particular recurring design problem that arises in specific design contexts and presents a well-proven architectural solution for the problem. The solution is specified by describing the roles of its constituent elements, their responsibilities and relationships, and the ways in which they collaborate. Like the choice of tactics, the choice of an architectural pattern has a profound effect on quality attributes—usually more than one.

Patterns typically comprise multiple design decisions and, in fact, often comprise multiple quality attribute tactics. We say that patterns often bundle tactics and, consequently, frequently make tradeoffs among quality attributes.

We will look at example relationships between tactics and patterns in each of our quality attribute-specific chapters. Chapter 14 explains how a set of tactics for any quality attribute can be constructed; those tactics are, in fact, the steps we used to produce the sets found in this book.